Clase 11 - Actividad: Publicar cuatro modelos en Hugging Face Models

1. ¿De qué forma subiste los archivos: web, VS Code o terminal? Si usaste un token de Hugging Face, ¿qué permiso le diste y dónde quedó guardado? ¿Por qué no hay que escribirlo en el código ni subirlo a GitHub?

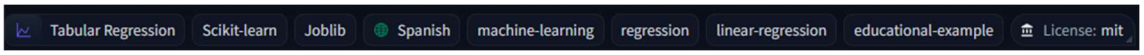
Directamente desde la web de Hugging Face mediante la opción Upload.

2. Mirá la pestaña Model card de los cuatro repos. ¿Qué etiquetas aparecen arriba en cada uno y de qué parte del README.md salen?

Se obtienen desde el encabezado **YAML** que se encuentra al inicio del archivo README.md.

01-sklearn-regresion

```yaml
---
language:
 - es
library_name: sklearn
pipeline_tag: tabular-regression
tags:
 - machine-learning
 - regression
 - linear-regression
 - educational-example
license: mit
---
```



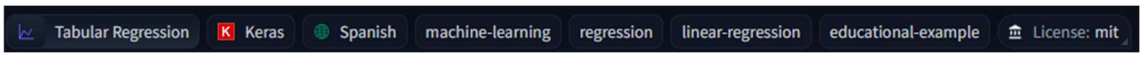
02-pytorch-regresion

```yaml
---
language:
 - es
library_name: pytorch
pipeline_tag: tabular-regression
tags:
 - machine-learning
 - regression
 - linear-regression
 - safetensors
 - educational-example
license: mit
---
```



03-keras-regresion

```
---
language:
 - es
library_name: keras
pipeline_tag: tabular-regression
tags:
 - machine-learning
 - regression
 - linear-regression
 - educational-example
license: mit
---
```



04-transformers-clasificacion-texto

```
---
language:
 - es
library_name: transformers
pipeline_tag: text-classification
tags:
 - machine-learning
 - text-classification
 - bert
 - educational-example
license: mit
---
```



3. En cada repo, ¿cuál de los archivos es el modelo entrenado? ¿Cuáles necesita alguien que solo quiere usar el modelo y cuáles sirven para reproducirlo?

Repositorio	Archivo del modelo entrenado	Para usar el modelo	Para reproducirlo
scikit-learn	model.joblib	model.joblib (el objeto entero con pesos, bias y scaler ya viene adentro)	train.py.
PyTorch	model.safetensors	model.safetensors + config.json (medias y desvíos) + modelo.py (la clase con la arquitectura)	train.py.
Keras	model.keras	model.keras (la arquitectura, los pesos y la normalización ya vienen adentro)	train.py.

Transformers	model.safetensors	model.safetensors + config.json (arquitectura de la red) + archivos del tokenizer (tokenizer.json y tokenizer_config.json)	train.py.

4. Los repos 02 (PyTorch) y 04 (Transformers) tienen un config.json. ¿Guardan lo mismo? ¿Qué guarda cada uno?

No guardan lo mismo:

- **PyTorch:** Guarda las columnas de entrada y los parámetros estadísticos de normalización (media y desvío de cada columna) para poder escalar los datos antes de pasárselos al modelo.
- **Transformers:** Es la configuración estándar oficial de la librería transformers. Guarda la arquitectura exacta de la red neuronal BERT.

5. ¿Por qué el repo de PyTorch necesita modelo.py y el de Keras no necesita ningún archivo parecido?

- **PyTorch (model.safetensors):** El modelo solo guarda un diccionario plano con los tensores de los pesos y sesgos (w y b) sin la arquitectura. Por lo tanto, Python necesita tener la definición de la clase RegresionLineal(torch.nn.Module) en modelo.py para instanciar la red y cargar esos pesos.
- **Keras (model.keras):** El formato .keras es un contenedor que guarda la arquitectura y los pesos. Al hacer cargar el modelo, Keras ya sabe reconstruir las capas sin necesidad de código Python auxiliar.

6. En el TP usamos InferenceClient con cardiffnlp/twitter-xlm-roberta-base-sentiment. ¿Se puede usar alguno de tus cuatro modelos de la misma forma? ¿Por qué?

De los cuatro modelos, el modelo basado en **Transformers** es el que podría utilizarse directamente con **InferenceClient** (siempre que sea compatible con alguno de los proveedores de inferencia de Hugging Face), porque Hugging Face ofrece integración con modelos Transformers mediante sus servicios de inferencia remota.

Los modelos de regresión desarrollados con **Scikit-learn, PyTorch y Keras** no quedan automáticamente disponibles para inferencia remota, porque necesitan desplegarse en un servicio de inferencia compatible o implementar una API propia que cargue el modelo y procese las predicciones.

7. Tocá *Use this model* en cada repo. ¿En cuáles aparece? ¿El código que propone funcionaría con tus archivos? Si en alguno no aparece o no funciona, ¿por qué te parece que pasa?

Aparece en el **01 (scikit-learn)**, en el **03 (Keras)** y en el **04 (Transformers)**.

- **04 (Transformers):** Funciona porque la librería transformers busca los nombres estándar (config.json, model.safetensors, tokenizer) que coinciden exactamente con lo subido.
- **03 (Keras):** Falla, porque el atajo **hf://** de Keras espera otra estructura de exportación en vez del archivo model.keras suelto.

- **01 (scikit-learn):** Falla porque asume que el archivo se llama sklearn_model.joblib, cuando en realidad se llama model.joblib.

No aparece **2 (PyTorch)** porque Hugging Face no conoce la clase o arquitectura de un modelo de PyTorch personalizado.

8. Si una compañera o un compañero quiere usar tu modelo de PyTorch, ¿qué tiene que tener en su código además de lo que baja del Hub?

Necesita tener la definición de la clase RegresionLineal para poder recrear la estructura de la red, como así también, la lógica de normalización con las medias y desvíos de config.json.

9. Además de Hugging Face, ¿dónde más se podría compartir un modelo? ¿Qué le faltaría a ese lugar comparado con el Hub?

En un repositorio de **GitHub**, **AWS S3, Google Cloud, Azure** o plataformas como **Kaggle.**

Les faltaría:

- **Model Cards:** Documentación integrada con metadatos, licencias y visualización de datasets.
- **Descubrimiento y comunidad:** Buscador global por tags de tareas (tabular-regression, text-classification, etc.).
- **Versionado de pesos grandes con LFS:** Manejo nativo y sin fricción de archivos binarios pesados.
- **Integración con la librería huggingface_hub:** Descarga y caché automática con una sola línea de código (hf_hub_download o from_pretrained).
- **Inference API & Spaces:** Facilidad para levantar una demo web con Gradio/Streamlit o probarlo en la nube con un clic.